

**Murilo Quadros Ortiz
João Vitor dos Santos Pereira**

RELATÓRIO TÉCNICO WEB PROXY COM CONTROLE DE CONTEÚDO

Rio Grande, 2026

1. Justificativa da Tecnologia Escolhida

A seleção do ecossistema de desenvolvimento, composto pela linguagem Python, o microframework Flask e a biblioteca HTTP *requests*, justificou-se primordialmente pelo equilíbrio entre a eficiência no desenvolvimento e o alinhamento com os objetivos pedagógicos da disciplina de Sistemas para Internet 2. A linguagem Python foi escolhida por sua sintaxe limpa e pela vasta gama de ferramentas voltadas à manipulação de strings e textos, característica essencial para um sistema cujo propósito envolve a filtragem e reestruturação de conteúdo HTML em tempo real.

No que tange à arquitetura de rede, a utilização do Flask revelou-se significativamente vantajosa quando comparada à implementação de um proxy via Sockets TCP puros em baixo nível. Embora a abordagem por sockets ofereça um desempenho bruto superior e menor *overhead* de processamento por não possuir camadas intermediárias, ela exigiria que a dupla gerenciasse manualmente os buffers de leitura, fizesse o *parsing* (interpretação) artesanal dos cabeçalhos HTTP e estruturasse individualmente cada pacote de resposta. O Flask, por sua vez, abstrai essa complexidade de infraestrutura de rede, tratando as requisições nativamente através de um sistema de rotas dinâmicas. Isso permitiu que o foco do desenvolvimento fosse direcionado à lógica de negócios do proxy — como as regras de bloqueio e substituição de conteúdo —, garantindo maior confiabilidade no tráfego de dados e reduzindo a incidência de falhas de protocolo.

Em contrapartida, ao analisar alternativas de nível equivalente ou superior, o Flask destacou-se por seu caráter minimalista e focado. Frameworks robustos como o Django introduziriam uma sobrecarga desnecessária de código e configuração para o escopo deste trabalho, uma vez que trazem acoplados sistemas de mapeamento objeto-relacional (ORM), controle de rotinas de autenticação e painéis administrativos que não possuem utilidade em um proxy de conteúdo. Já em relação a ferramentas modernas como o FastAPI, que se destaca pelo suporte nativo a operações assíncronas e alta performance sob múltiplas requisições simultâneas, o Flask apresentou-se como uma escolha mais pragmática por ser a tecnologia base solidificada em sala de aula. Essa escolha mitigou a curva de aprendizado inicial da dupla em relação ao gerenciamento de microsserviços web, viabilizando a entrega de um software estável, bem estruturado e de fácil manutenção dentro do prazo estipulado.

2. Decisões de Implementação Relevantes

No âmbito da arquitetura do software, uma das decisões de implementação mais relevantes foi a adoção de um design modular, segmentando o fluxo de execução do proxy em funções com responsabilidades únicas e bem definidas. Foram estruturadas sub-rotinas específicas para o carregamento dinâmico dos arquivos de configuração em formato JSON, para a persistência de históricos no arquivo de logs, para a renderização da página customizada de interdição e para a execução do algoritmo de varredura e substituição de termos proibidos. Essa organização abstrata não apenas refinou a legibilidade e a manutenibilidade do código-fonte, mas também facilitou o diagnóstico de falhas durante o desenvolvimento, permitindo que a lógica de manipulação de strings fosse testada de forma isolada da camada de transporte de dados do Flask.

A principal complexidade técnica enfrentada pela dupla residiu no processo de roteamento e na extração fidedigna do domínio alvo a partir da URL encapsulada na requisição do cliente. Como o proxy opera recebendo o endereço de destino como um parâmetro adaptado de sua própria

rota, tornou-se um desafio tratar as variações de formatos que podem ser introduzidas (como a presença ou ausência de prefixos de protocolo e subdomínios, a exemplo de `http://` ou `www.`). Foi necessário implementar uma lógica rigorosa de decomposição sintática da URI (*parsing*) para isolar estritamente o nome do *host* e, consecutivamente, efetuar a validação cruzada com a lista negra configurada no arquivo de bloqueios. Adicionalmente, garantir que o conteúdo correto fosse requisitado ao servidor de origem e repassado integralmente ao navegador exigiu um entendimento aprofundado sobre o ciclo de requisição-resposta do protocolo HTTP.

Por outro lado, o comportamento que se revelou mais surpreendente e desafiador durante os testes práticos envolveu a codificação de caracteres (*encoding*). Ao interceptar e renderizar páginas web em português, constatou-se que caracteres acentuados e cedilhas eram frequentemente corrompidos no navegador final, gerando falhas visuais na interface. Esse fenômeno ocorria porque a biblioteca *requests*, por padrão, adota codificações genéricas quando o servidor de origem não especifica explicitamente o *charset* nos cabeçalhos HTTP. A solução para essa limitação inesperada foi a implementação da propriedade `apparent_encoding`. Essa ferramenta analisa heurísticamente os bytes reais do corpo da resposta para decodificar o HTML de forma precisa antes de submetê-lo ao filtro de palavras, assegurando a integridade textual e visual do site entregue ao usuário.

3. Análise do Comportamento Frente ao Protocolo HTTPS e Limitações de Interceptação

O protótipo de proxy desenvolvido neste trabalho apresenta uma limitação estrutural clássica: sua capacidade de operação é restrita a páginas que utilizam o protocolo HTTP convencional. Quando o usuário tenta acessar um endereço que opera sob o protocolo HTTPS, a comunicação entre o navegador cliente e o servidor de destino passa a ser protegida pelas camadas de segurança TLS/SSL (*Transport Layer Security* / *Secure Sockets Layer*). Enquanto no HTTP os dados trafegam em texto claro (*cleartext*), permitindo que o proxy intercepte a requisição, leia a estrutura do HTML e aplique as substituições de strings de forma direta, o HTTPS estabelece uma criptografia de ponta a ponta. Como consequência, o *payload* (corpo da mensagem) que transita pelo proxy torna-se indecifrável, manifestando-se apenas como um fluxo de bytes cifrados cuja chave de decodificação é de posse exclusiva das duas extremidades legítimas da comunicação.

Para que o mecanismo de inspeção e filtragem de conteúdo proposto fosse capaz de atuar sobre conexões HTTPS, seria indispensável a implementação de uma arquitetura de interceptação SSL, conhecida tecnicamente como ataque controlado de *Man-in-the-Middle* (MitM). Sob essa abordagem, o proxy deixaria de ser um mero repassador de dados e assumiria o papel de um terminador de conexões duplo. Ele precisaria estabelecer uma sessão criptografada segura com o servidor de origem e, simultaneamente, abrir uma segunda sessão TLS com o navegador do cliente. Para viabilizar a comunicação com o cliente sem romper o protocolo, o proxy teria que gerar falsificações dinâmicas de certificados digitais para cada domínio solicitado. Essa técnica introduz uma complexidade operacional severa e exige uma quebra nos pressupostos de segurança do usuário, uma vez que o navegador bloquearia sumariamente os acessos a menos que uma Autoridade Certificadora (CA) raiz, gerada pelo próprio proxy, fosse previamente instalada e configurada como confiável no repositório do sistema operacional do cliente.

Como alternativa de menor complexidade para contornar o bloqueio total de conexões HTTPS — embora sem resolver o problema da filtragem —, o sistema poderia implementar o suporte ao método HTTP `CONNECT`. Esta especificação altera a dinâmica de atuação do proxy: ao receber uma requisição `CONNECT`, o servidor Flask não tentaria ler ou interpretar o cabeçalho da aplicação; em vez disso, ele atuaria estritamente na camada de transporte, estabelecendo um túnel TCP bidirecional e cego entre o cliente e o servidor remoto. Nessa modalidade de tunelamento, os dados cifrados são repassados de forma íntegra e transparente, permitindo que o usuário navegue por portais HTTPS sem alertas de violação de segurança. Contudo, essa abordagem anula o propósito central de controle de conteúdo do projeto, visto que o proxy permaneceria completamente agnóstico e incapaz de inspecionar ou expurgar os termos proibidos do fluxo de HTML trafegado.

4. Relato de Uso de Inteligência Artificial no Desenvolvimento do Projeto

Em consonância com as práticas modernas de engenharia de software e transparência acadêmica, o desenvolvimento deste projeto contou com o suporte estratégico de ferramentas de Inteligência Artificial Generativa. A dupla utilizou os modelos Claude (da Anthropic) e Gemini (da Google) não como substitutos do esforço intelectual, mas como assistentes interativos de co-programação, facilitadores pedagógicos e revisores textuais. Como a dupla já possuía uma base sólida e domínio prévio nas linguagens Python, HTML e JavaScript, as lacunas de conhecimento concentravam-se na utilização e configuração do framework Flask, tecnologia com a qual os componentes do grupo ainda não haviam trabalhado extensivamente.

O modelo Claude foi integrado logo na fase inicial do ciclo de desenvolvimento do software. Sua atuação concentrou-se no esclarecimento de dúvidas conceituais sobre a arquitetura do Flask e na sugestão de bibliotecas nativas e de terceiros que fossem ideais para a manipulação de requisições web. A ferramenta foi responsável por gerar o esqueleto sintático inicial do servidor (`app.py`), ensinando à dupla como mapear rotas coringa dinâmicas capazes de capturar URLs completas como argumentos através da diretiva `/<path:url_alvo>`. Adicionalmente, o Claude sugeriu o uso da biblioteca `requests` para gerenciar as chamadas externas HTTP de forma limpa e o uso de expressões regulares através da biblioteca nativa `re` (especificamente a função `re.subn`) como o mecanismo mais performático para realizar a varredura e a contagem de palavras substituídas no HTML.

A partir do código base e das recomendações estruturais propostas pela IA, a dupla realizou modificações profundas e implementações autorais para moldar o sistema às regras de negócios específicas do trabalho. Coube aos desenvolvedores projetar e programar a lógica de persistência e leitura de dados dos arquivos de configuração local (`blocked.json` e `words.json`), bem como estruturar o histórico incremental de auditoria armazenado em `log.json`. A dupla também criou a interface visual da página de interdição, inserindo código HTML/CSS diretamente encapsulado na lógica do proxy, e resolveu de forma autônoma os problemas de decodificação de caracteres ao integrar a propriedade `apparent_encoding` do pacote `requests` após observar falhas na renderização de páginas em português.

Na etapa de encerramento do projeto, o modelo Gemini foi adotado com foco exclusivo em engenharia de documentação e refinamento de linguagem. A ferramenta foi empregada para analisar o código final consolidado e traduzir as minúcias técnicas complexas em uma

documentação explicativa e acessível dentro do arquivo `README.md`, permitindo que usuários leigos ou administradores sem conhecimento em programação compreendam o funcionamento do proxy. O Gemini também atuou como um revisor ortográfico, gramatical e estrutural para o relatório técnico, garantindo a coesão textual e o alinhamento com o rigor linguístico exigido em produções de nível universitário.

A interação com as Inteligências Artificiais propiciou um aprendizado valioso para a dupla, indo além da simples entrega do software funcional. Os desenvolvedores absorveram conceitos fundamentais sobre o ciclo de requisição e resposta do Flask, compreenderam o comportamento e a manipulação de parâmetros de caminhos (*path parameters*) em URLs e aprenderam a avaliar criticamente as sugestões de uma IA, adaptando-as à realidade prática de um sistema de redes. Sobretudo, a experiência consolidou a habilidade dos acadêmicos de utilizar modelos de linguagem como tutores dinâmicos, acelerando o domínio de novas tecnologias de forma ética, consciente e transparente.

5. Aprendizados Adquiridos sobre o Protocolo HTTP

A implementação prática do servidor proxy consolidou o entendimento da dupla sobre a natureza estrita e puramente textual do protocolo HTTP, evidenciando como a comunicação na camada de aplicação depende do cumprimento rigoroso de regras de sintaxe e semântica. Ficou claro que o ecossistema da Web opera baseado em um ciclo previsível de requisição e resposta, no qual cada ação tomada por um intermediário deve ser formalmente comunicada ao cliente através de metadados padronizados. A experiência de interceptar esse fluxo permitiu visualizar de forma concreta como os navegadores interpretam as mensagens enviadas pelos servidores e como pequenas divergências na formatação dessas respostas podem interromper o carregamento de uma página ou comprometer a experiência do usuário.

Esse aprendizado tornou-se evidente na estruturação do mecanismo de interdição de domínios. A dupla compreendeu que interromper o acesso a uma página restrita exige mais do que apenas cessar o envio de dados ou encaminhar uma interface gráfica customizada de alerta. Para que o navegador interprete a ação corretamente e trate o evento de forma semântica, o proxy necessita emitir explicitamente o código de status HTTP adequado — neste caso, o código `403 Forbidden` —, encapsulado na resposta juntamente com o corpo do arquivo HTML de bloqueio. Essa dinâmica evidenciou a importância dos códigos de estado na governança de redes, demonstrando como eles ditam o comportamento dos clientes na Web.

Adicionalmente, o projeto propiciou um domínio prático a respeito da utilidade e manipulação dos cabeçalhos HTTP (*headers*), com destaque para o campo `Content-Type`. A dupla assimilou que esse cabeçalho atua como o principal descritor de mídia da mensagem, instruindo o receptor se o payload transportado consiste em um documento estruturado HTML, em uma imagem, em um arquivo de estilização ou em dados binários genéricos. No contexto do proxy, a validação sistemática do `Content-Type` mostrou-se indispensável para a otimização e viabilidade do sistema. Foi esse cabeçalho que permitiu ao algoritmo diferenciar requisições de páginas web (onde a varredura e substituição por expressões regulares no texto são necessárias) de requisições de outros ativos digitais, garantindo que conteúdos não-textuais fossem repassados de forma íntegra e sem modificações que pudessem corromper os arquivos originais.